

Skewed Datasets (Optional)

A **skewed dataset** is a classification problem where the ratio of positive (1) to negative (0) examples is very far from 50/50.

- **Example:** Rare disease detection, where only 0.5% of patients have the disease ($y = 1$) and 99.5% do not ($y = 0$).
-

The Problem with Classification Error

Using **classification error** (or **accuracy**) as your only metric is highly misleading for skewed datasets.

- **Example:** If the disease is present in only 0.5% of the population, a "dumb" (and useless) algorithm that *always* predicts $y = 0$ will have:
 - **0.5% error**
 - **99.5% accuracy**
 - If your complex machine learning model achieves a 1% error (99% accuracy), it *looks* good, but it's performing *worse* than the useless algorithm that just guesses "no" every time.
 - This shows that accuracy is not a good metric for telling if your model is actually useful.
-

Better Metrics: Precision and Recall

For skewed datasets, we use two different metrics: **Precision** and **Recall**. To calculate them, we first build a **Confusion Matrix**.

The Confusion Matrix

A confusion matrix is a 2x2 table that breaks down your model's predictions on a test set (or CV set) into four categories:

	Actual Class: 1 (Has Disease)	Actual Class: 0 (No Disease)
Predicted Class: 1	True Positive (TP) <i>Correctly predicted "yes"</i>	False Positive (FP) <i>Wrongly predicted "yes"</i>
Predicted Class: 0	False Negative (FN) <i>Wrongly predicted "no"</i>	True Negative (TN) <i>Correctly predicted "no"</i>

Precision

- **Question:** Of all the times the model predicted a patient **has the disease** ($y = 1$), what fraction of those patients **actually** had the disease?
- **Intuition:** "When my model says 'yes', how often is it right?" (This is a measure of *trustworthiness*).

- **Formula:**

$$Precision = \frac{\text{True Positives}}{\text{Total Predicted Positives}} = \frac{TP}{TP+FP}$$

Recall

- **Question:** Of all the patients who **actually had the disease**, what fraction did the model **correctly detect**?
- **Intuition:** "Of all the 'yes' cases that actually exist, how many did my model find?" (This is a measure of *completeness*).

- **Formula:**

$$Recall = \frac{\text{True Positives}}{\text{Total Actual Positives}} = \frac{TP}{TP+FN}$$

Using Precision and Recall

- The "dumb" algorithm that *always* predicts $y = 0$ would have 0 True Positives (TP).
- Its **Recall would be 0%** ($\frac{0}{0+FN}$).
- This metric immediately and correctly tells you the algorithm is useless.
- A good algorithm must achieve **both** high precision and high recall.

The Precision-Recall Trade-off (Optional)

Ideally, we want a model to have both high precision and high recall. However, in practice, these two metrics are often in conflict: improving one can hurt the other. This relationship is managed by adjusting the model's **classification threshold**.

The Role of the Threshold

Recall that logistic regression outputs a probability $f(x)$ (a number between 0 and 1). We typically use a **threshold of 0.5** to make a prediction:

- If $f(x) \geq 0.5$, predict $\hat{y} = 1$
- If $f(x) < 0.5$, predict $\hat{y} = 0$

We can *change* this threshold to prioritize either precision or recall, depending on our goal.

Two Scenarios for Adjusting the Threshold

Scenario 1: Prioritizing High Precision (Being "Confident")

- **Goal:** We want to be very sure that when the model predicts $\hat{y} = 1$, it is correct. We want to avoid **False Positives (FP)**.
- **Example:** Predict $\hat{y} = 1$ (has disease) only if a treatment is very expensive, invasive, or risky. We don't want to treat healthy people.
- **Action: Raise the threshold** (e.g., to 0.7 or 0.9).
- **Result:**
 - **Higher Precision:** The model only predicts 1 when it's highly confident, so it makes fewer False Positives.
 - **Lower Recall:** The model will be too cautious and miss more of the actual positive cases (it will have more **False Negatives, FN**).

Scenario 2: Prioritizing High Recall (Being "Safe")

- **Goal:** We want to find *all* possible positive cases. We want to avoid **False Negatives (FN)**.
- **Example:** Predict $\hat{y} = 1$ (has disease) if a disease is very dangerous if *not* treated, but the treatment itself is cheap and safe. We'd rather treat 10 healthy people than miss one sick person.
- **Action: Lower the threshold** (e.g., to 0.3).
- **Result:**
 - **Higher Recall:** The model predicts 1 more readily, so it correctly identifies more of

the actual positive cases.

- **Lower Precision:** The model will also incorrectly label more negative cases as positive (it will have more **False Positives, FP**).

This relationship can be visualized on a **Precision-Recall Curve**, which shows the trade-off as you vary the threshold.

The F1-Score: Combining Metrics

Since we now have two metrics, how do we compare models?

- Model A: High Precision, Low Recall
- Model B: Low Precision, High Recall

We need a single number that combines both. A simple **average** $((P+R)/2)$ is **not a good metric** because it can be "gamed" by a useless model (e.g., a model with 100% recall but 1% precision would look good).

The standard solution is the **F1-Score**.

- **What it is:** The **harmonic mean** of Precision and Recall.
- **Key Property:** The F1-Score gives more weight to the *lower* of the two values. It will only be high if **both** precision and recall are high.
- **Formula:**

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

By calculating the F1-Score, you can get a single, more reliable metric to compare different algorithms or threshold choices for your skewed dataset.